Министерство цифрового развития, связи и массовых коммуникаций РФ
Уральский технический институт связи и информатики (филиал)
ФГБОУ ВО «Сибирский государственный университет
телекоммуникаций и информатики» в г. Екатеринбурге
(УрТИСИ СибГУТИ)

Отчёт
по учебной практике
по МДК 02.02 «Инструментальные средства разработки
программного обеспечения»
на тему «Разработка веб-приложения»

студента IV курса 285 группы

Фамилия Сесин

Имя, отчество Даниил Алексеевич

Факультет Инфокоммуникаций, информатики и управления

Специальность 09.02.07 «Информационные системы
и программирование»

Екатеринбург, 2026

Отзыв руководителя

Содержание

Введение....4
1 Учебная практика....5
1.1 Анализ предметной области....5
1.2 Выбор стека технологий и инструментов разработки....6
1.3 Разработка базы данных....7
1.4 Практическая реализация приложения....10
1.5 Разработка веб-приложения....12
1.6 Тестирование программы....15
1.7 Качественная характеристика кода....16
2 Дипломный проект....17
2.1 Анализ предметной области....17
2.2 Разработка дополнительного функционала....18
2.3 Разработка функции быстрого выбора VR-игры в веб-приложении....19
2.4 Разработка функции статистики просмотра VR-игр в веб-приложении21
Заключение....23
Библиография....24
Приложение А....25

Введение

Данная учебная практика направлена на подготовку к демонстрационному экзамену и описанию практической части дипломного проекта.

Цель учебной практики – проработать вариант демонстрационного экзамена и составить пояснительную записку к практической части дипломного проекта.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать техническое задание и предметную область;
- сформировать базу данных и архитектуру программы;
- разработать приложение;
- разработать документацию к продукту;
- протестировать программу;
- проработать приложение дипломного проекта.

1 Учебная практика

1.1 Анализ предметной области

Разрабатываемое приложение предназначено для автоматизации учета, сопровождения и контроля заявок на ремонт климатической техники в сервисной организации ООО «Конди».

В соответствии с техническим заданием система должна обеспечивать выполнение следующих функциональных требований:

— создание и сохранение заявок на ремонт в базе данных;

— просмотр и изменение информации по уже зарегистрированным заявкам;

— контроль текущего состояния заявки на всех этапах выполнения работ;

— назначение специалиста, ответственного за выполнение ремонта;

— ведение комментариев и служебных заметок по каждой заявке;

— учет комплектующих и дополнительных сведений, связанных с ремонтом;

— формирование статистических данных по работе сервиса;

— контроль просроченных заявок и обращений, требующих дополнительного внимания;

— предоставление QR-кода для перехода к форме оценки качества обслуживания.

Помимо функциональных возможностей, программа должна удовлетворять следующим нефункциональным требованиям:

— надежность хранения и обработки данных;

— защита доступа к системе с учетом ролей пользователей;

— удобный и понятный пользовательский интерфейс;

— приемлемая производительность при работе со списками заявок и отчетами;

— возможность дальнейшего расширения и доработки приложения.

Отдельно стоит отметить, что при разработке было необходимо выбрать такие средства реализации, которые позволили бы создать удобное веб-приложение с поддержкой базы данных, авторизации пользователей и разграничения прав доступа.

Входными данными для программы являются:

— данные учетных записей пользователей – логин, пароль, ФИО, телефон;

— сведения, вводимые при создании заявки – тип оборудования, модель устройства, описание неисправности, данные клиента;

— информация, добавляемая в ходе обработки заявки – статус ремонта, назначенный специалист, комментарии, перечень комплектующих, отметки о завершении;

— служебные записи, связанные с запросами помощи или контролем качества.

Выходными данными являются:

— список заявок с возможностью поиска и фильтрации по статусу, типу оборудования и другим параметрам;

— карточка заявки с полной информацией о ремонте;

— системные уведомления об успешном выполнении операций или об ошибках;

— статистические сведения по количеству заявок, срокам выполнения и наиболее частым неисправностям;

— данные для контроля просроченных заявок и обращений в модуле качества;

— сгенерированный QR-код для оценки качества обслуживания.

1.2 Выбор стека технологий и инструментов разработки

Для реализации программного обеспечения выбран стек Python с использованием фреймворка Flask. В качестве среды разработки применялся Visual Studio Code, который обеспечивает удобное редактирование, запуск и отладку приложения. Python и Flask были выбраны за простоту разработки, устойчивую работу на разных ОС и большое количество готовых библиотек.

Веб-интерфейс реализован на Flask с использованием шаблонов Jinja2. Такой подход позволяет разделить представление и бизнес-логику, упростить обработку форм, отображение данных и построение страниц.

В качестве системы управления базами данных используется SQLite. Она обеспечивает надёжное хранение данных и поддерживает транзакции. Для администрирования базы используются стандартные средства SQLite (файл `service_desk.db`), что упрощает резервное копирование и перенос.

Доступ к данным реализован через Flask-SQLAlchemy. Эта библиотека позволяет работать с данными на уровне объектов Python, без ручного написания SQL-запросов. Модели связываются с таблицами базы данных и поддерживают автоматическое формирование запросов и проверку ограничений.

Механизмы аутентификации и авторизации реализованы на базе Flask-Login. При входе пользователя сохраняется сессия, а проверка доступа выполняется через роль пользователя (`admin`, `operator`, `master`, `manager`) и декораторы доступа. Такой подход позволяет разграничивать права, защищать разделы системы и безопасно работать с заявками, комментариями и данными клиентов.

1.3 Разработка базы данных

Перед созданием базы данных были проанализированы требования и выделены следующие сущности и их атрибуты:

1) roles (роли):
   - role_id (PK);
   - role_name;
2) users (пользователи):
   - user_id (PK);
   - login;
   - password_hash;
   - role_id (FK);
   - master_id (FK, необязательно);
3) masters (специалисты):
   - master_id (PK);
   - full_name;
   - specialization;
   - phone;
4) clients (клиенты):
   - client_id (PK);
   - full_name;
   - phone;
   - email;
   - address;
5) equipment (оборудование):
   - equipment_id (PK);
   - model;
   - serial_number;
   - client_id (FK);
   - equipment_type;
6) requests (заявки):
   - request_id (PK);
   - create_date;
   - description;
   - equipment_id (FK);
   - master_id (FK, необязательно);
   - status (open, in_progress, waiting_parts, completed);
   - completion_date;
   - deadline_date;
   - customer_approved_extension;
7) fault_types (типы неисправностей):

- fault_type_id (PK);
- fault_name;

8) request_details (детали заявки):
- detail_id (PK);
- request_id (FK);
- fault_type_id (FK);

9) request_comments (комментарии):
- comment_id (PK);
- request_id (FK);
- user_id (FK);
- comment_text;
- created_at;

10) ordered_parts (комплектующие):
- part_id (PK);
- request_id (FK);
- part_name;
- quantity;
- notes;
- ordered_at;

11) request_collaborators (привлеченные специалисты):
- collaboration_id (PK);
- request_id (FK);
- master_id (FK);
- added_at;

12) status_notifications (уведомления о смене статуса):
- notification_id (PK);
- request_id (FK);
- user_id (FK);
- old_status;
- new_status;
- message;
- created_at;

13) feedback_reviews (отзывы):
- review_id (PK);
- request_id (FK, необязательно);
- rating;
- feedback_text;
- created_at.

Связи между сущностями:
- роли и пользователи (один ко многим);
- клиенты и оборудование (один ко многим);
- оборудование и заявки (один ко многим);

– специалисты и заявки (один ко многим);
– заявки и типы неисправностей (многие ко многим через request_details);
– заявки и комментарии (один ко многим);
– заявки и комплектующие (один ко многим);
– заявки и уведомления (один ко многим);
– заявки и привлеченные специалисты (многие ко многим через request_collaborators);
– заявки и отзывы (один ко многим).

ERD-диаграмма системы представлена в рисунке 1.1.

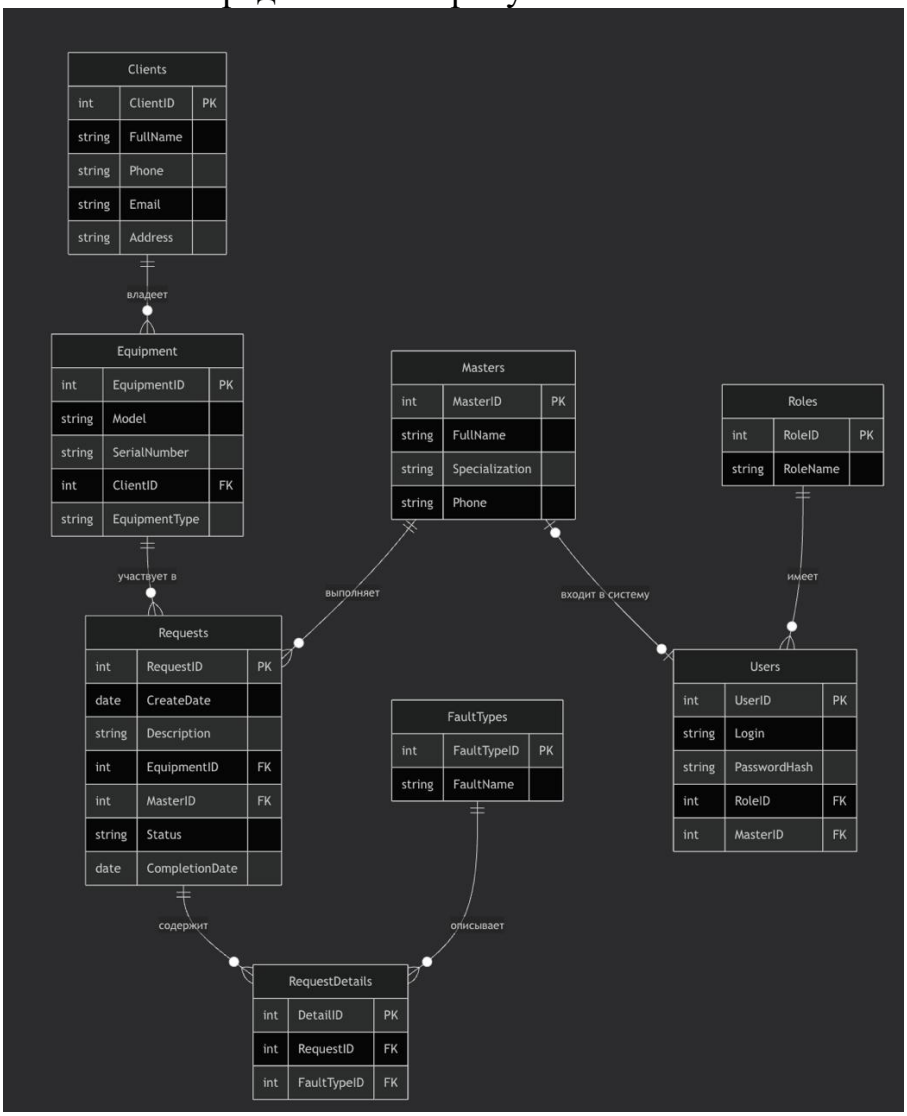

Рисунок 1.1 – ER-диаграмма

Запросы создания и заполнения базы данных представлены в приложении А.

1.4 Практическая реализация приложения

Перед разработкой программного продукта была составлена блок-схема основного алгоритма работы системы. На данной схеме отражена последовательность действий пользователя при входе в приложение, проверке корректности учетных данных, определении роли и дальнейшем выполнении доступных операций. После успешной авторизации пользователь получает доступ к главному окну системы, где может выполнять действия в соответствии со своей ролью: создавать, редактировать и просматривать заявки, добавлять комментарии, анализировать статистику и завершать работу с системой. Блок-схема позволяет наглядно представить общую логику функционирования приложения и порядок перехода между основными этапами работы.

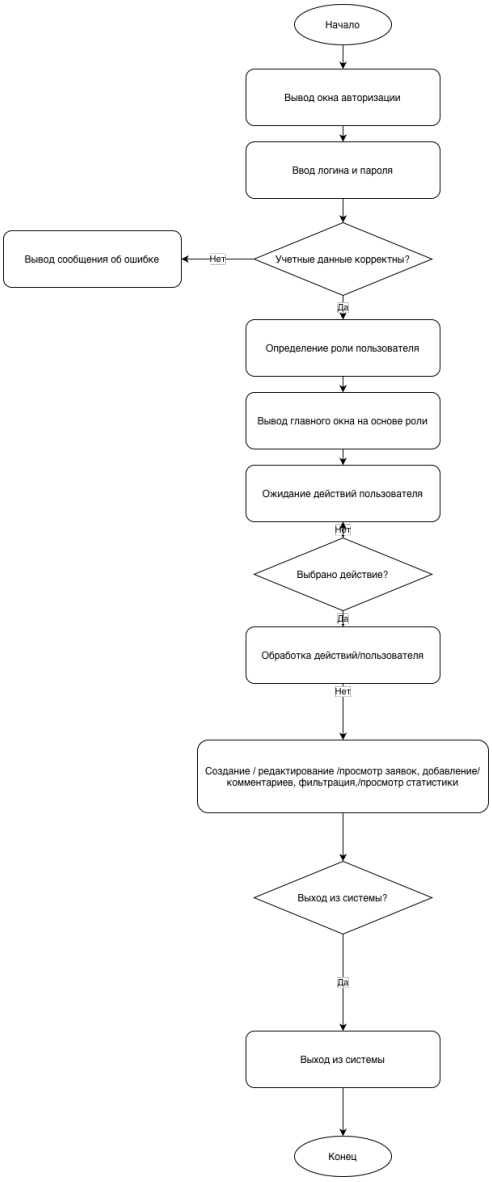

Рисунок 1.2 – Основной алгоритм работы системы

Программное обеспечение реализовано в виде веб-приложения на Python с использованием Flask. Структура проекта разделена на уровни данных, бизнес-логики, интерфейса и служебных компонентов.

Компоненты системы:

– Уровень данных: структура базы данных описана в `app/models.py` с применением SQLAlchemy. Сущность заявки связана с оборудованием и специалистом, а комментарии связаны с заявкой и пользователем;

– Уровень доступа к данным: взаимодействие с БД выполняется через Flask-SQLAlchemy, модели связаны с таблицами, ограничения и проверки заданы на уровне схемы;

– Уровень представления: пользовательский интерфейс реализован на шаблонах Jinja2 (`app/templates`). Интерфейс включает список заявок, формы создания и редактирования, карточку заявки, статистику и раздел качества;

– Уровень логики: маршруты и обработчики находятся в `app/routes.py`. Здесь выполняются проверки прав доступа, загрузка и сохранение данных, обработка действий пользователя;

– Разделение по ролям: в системе используются роли `admin`, `operator`, `master`, `manager`. В зависимости от роли пользователю открываются только разрешённые функции;

– Действия по ролям: оператор и менеджер создают и изменяют заявки, назначают специалистов; мастер обновляет статус, добавляет комментарии и фиксирует комплектующие; менеджер по качеству привлекает специалистов и продлевает сроки; администратор управляет пользователями и ролями.

Вход пользователя реализован через страницу авторизации. После успешной проверки логина и пароля формируется пользовательская сессия (Flask-Login), а доступ к разделам ограничивается декоратором `role_required` и проверками роли.

Работа с заявками реализована в разделе заявок и включает:

– список заявок: поиск, фильтрация по статусу и специалисту, сортировка по дате;

– карточка заявки: полная информация о заявке, клиенте, оборудовании, назначенном специалисте, комментариях и комплектующих;

– создание заявки: автоматическая фиксация даты создания и статуса;

– редактирование заявки: доступные поля зависят от роли;

– удаление заявки: доступно только администратору или менеджеру (при необходимости можно ограничить/отключить).

Раздел комментариев позволяет специалистам фиксировать результаты диагностики и ремонта, а также внутренние замечания по заявке. Менеджер по качеству может привлекать дополнительных специалистов и продлевать сроки выполнения.

Раздел статистики предназначен для расчёта и отображения ключевых показателей:

– количество выполненных заявок;
– среднее время выполнения;
– статистика по типам неисправностей.

Дополнительно реализованы средства повышения удобства работы:
– обработка ошибок доступа и отсутствующих данных;
– уведомления об успешных действиях и ошибках;
– генерация QR-кода для перехода к форме оценки качества обслуживания.

Таким образом, практическая реализация программы охватывает основные функции системы учета заявок на ремонт климатического оборудования и обеспечивает работу пользователей в соответствии с их ролями и полномочиями.

1.5 Разработка веб-приложения

В результате разработки программы реализовано следующее:
– меню вкладок с таблицами;
– права доступа согласно ролям;
– авторизация по логину и паролю;
– CRUD операции;
– обработка ошибок и предупреждений при удалении данных;
– вкладка со статистикой работы отдела;
– отображение данных текущего пользователя и функция выхода из системы.

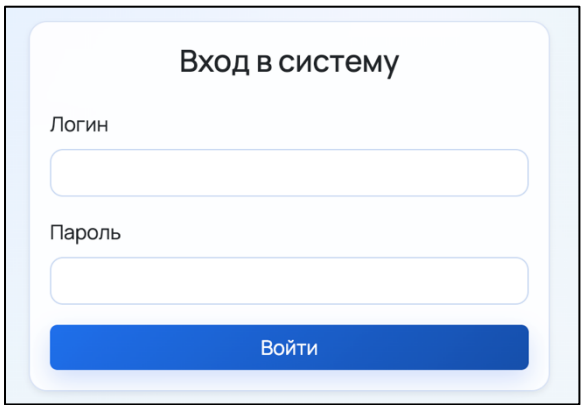

Рисунок 1.4 – Окно авторизации

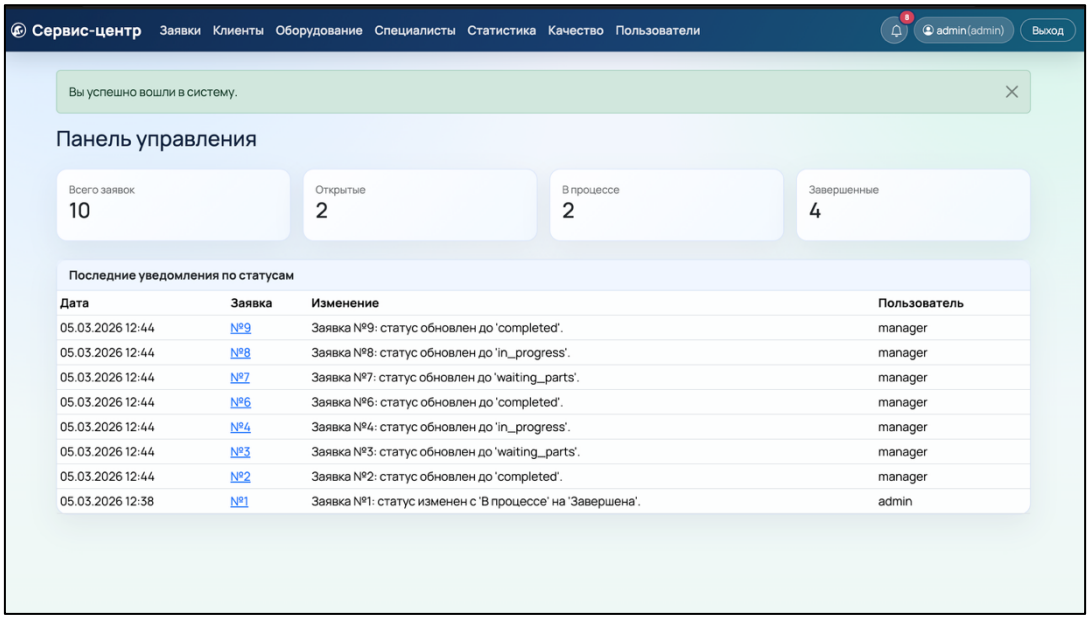

Рисунок 1.5 – Главная вкладка

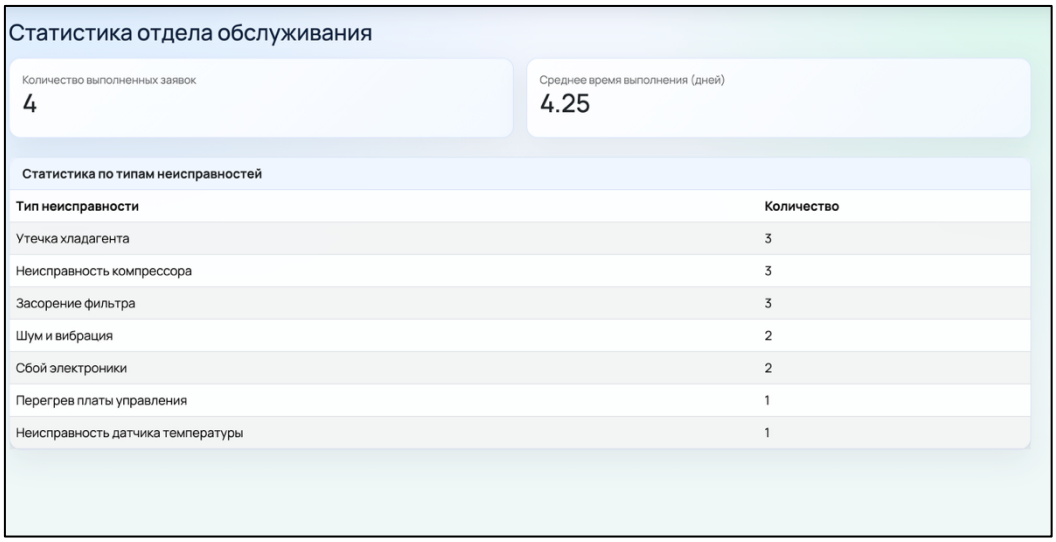

Рисунок 1.6 – Окно статистики

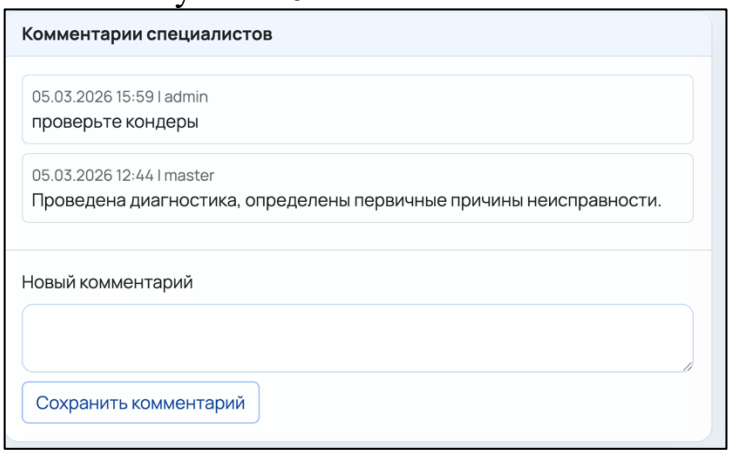

Рисунок 1.7 – Окно комментариев

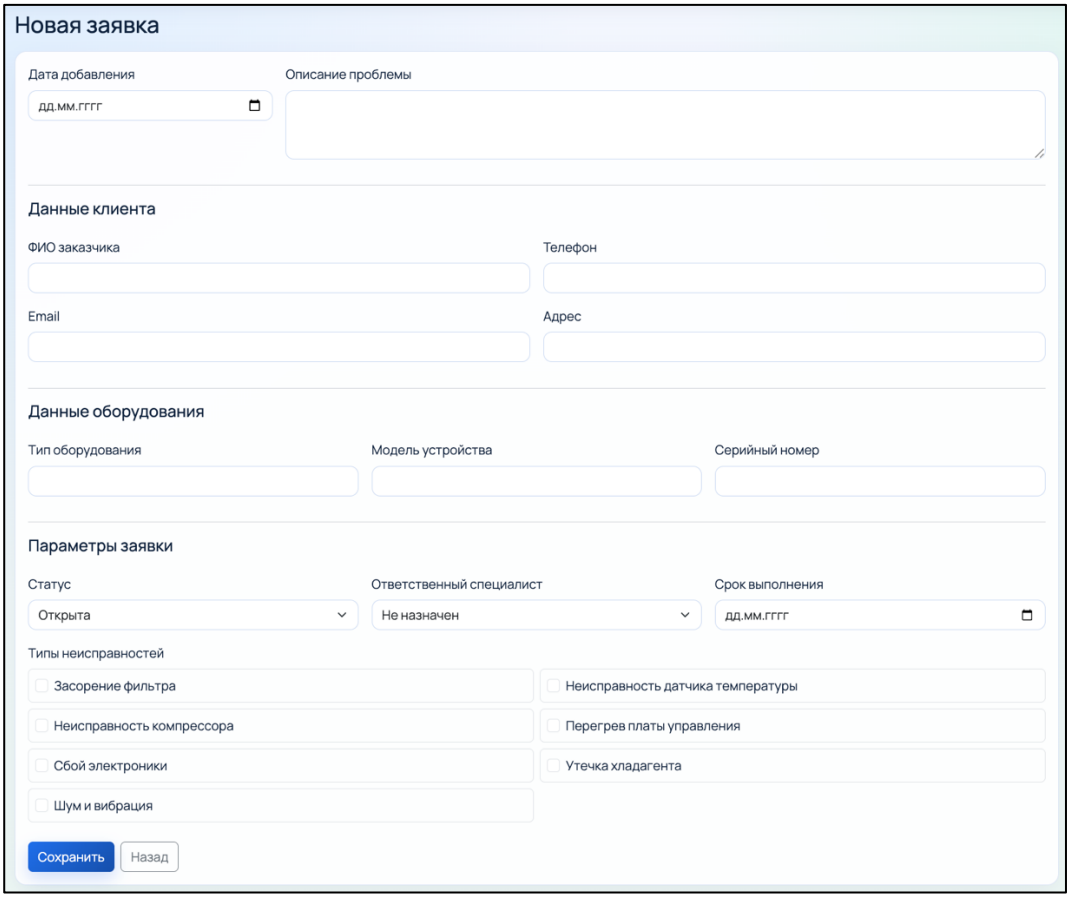

Рисунок 1.8 – Создание заявки

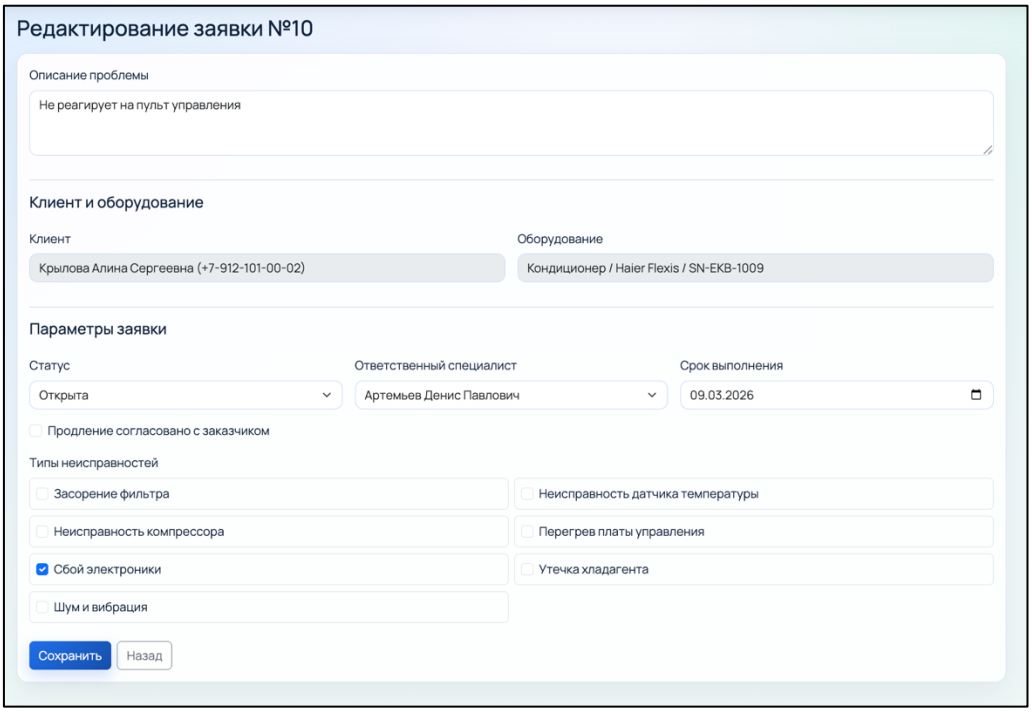

Рисунок 1.9 – Редактирование заявки

1.6 Тестирование программы

После реализации программы она была протестирована, тесты и их результаты представлены в таблице 1.1.

Таблица 1.1 – Тестирование программы

№ теста	Модуль тестирования	Порядок тестирования	Ожидаемый результат	Результат теста
1	Логин	Ввод корректного пароля и логина	Вход в систему	Успешный вход
2	Логин	Ввод неправильного пароля или логина	Появление ошибки	Вывод ошибки без входа в систему
3	Ролевой доступ	Поочерёдный вход в систему под разными ролями пользователей	Успешное отображение таблиц в соответствии с ролями	У каждой роли отображается свой функционал

Таблица 1.2 – Тестирование программы

4	Работа с данными	Создать заявку	Заявка успешно создана	Успешно созданная заявка
5	Работа с данными	Удалить заявку	Заявка удалена	Успешно удаленная заявка
6	Работа с данными	Изменение заявки	Заявка изменена	Успешно измененная заявка
7	Работа с данными	Добавление комментария	Появление комментария	Не созданный комментарий к заявке
8	Работа с данными	Создание заявки с пустыми полями	Сообщение о пустых полях	Отображение ошибки о некорректности введенных данных

1.7 Качественная характеристика кода

После тестирования была выполнена оценка качества программного кода. Определение качественных характеристик кода: полнота обработки ошибочных данных включает проверки на отсутствие записей в базе данных, проверки корректности введённых данных, проверку пустых строк, проверку прав доступа по ролям, а также обработку ошибок при сохранении изменений в базе данных; однако отсутствуют централизованный механизм логирования ошибок, не все возможные исключительные ситуации обрабатываются отдельно, часть проверок реализована только на базовом уровне; таким образом, обработка ошибок реализована, но может быть дополнительно улучшена.

Автоматические тесты для проверки допустимых значений входных данных в проекте отсутствуют. Что касается средств контроля корректности входных данных, то присутствуют проверка модели, контроль заполнения обязательных полей, проверка ролей пользователей, проверка допустимости действий над заявками и комментариями, а также ограничения на доступ к отдельным функциям; однако не для всех полей реализована расширенная валидация формата и длины значений; таким образом, базовые средства контроля корректности входных данных присутствуют. Средства восстановления при сбоях оборудования в приложении отсутствуют: не реализованы автоматическое резервное копирование базы данных, журналирование с восстановлением, повторное выполнение операций при сбоях; следовательно, встроенные средства восстановления после сбоев отсутствуют. По наличию комментариев можно отметить, что структура кода и имена классов, свойств и методов в целом понятны, отдельные части кода снабжены пояснениями, однако подробное документирование реализовано не во всех файлах; таким образом, код в целом читаемый, но количество комментариев можно увеличить в наиболее важных участках.

Проверка корректности передаваемых данных выполняется: присутствуют проверка идентификаторов записей, существования заявок и пользователей, а также проверка прав на просмотр, изменение и удаление данных; итогом является то, что проверка корректности передаваемых данных реализована. Описания основных функций присутствуют: основные функции имеют понятные названия и логичное разделение по разделам приложения, например создание заявки, редактирование, просмотр списка, работа со статистикой, комментариями и авторизацией; таким образом, основные функции программы выделены и читаются достаточно понятно.

2 Дипломный проект

2.1 Анализ предметной области

Парк виртуальной реальности WARPOINT Veer Mall является современным развлекательным пространством, предоставляющим посетителям возможность погружения в интерактивные игровые миры с использованием технологий виртуальной реальности. Основное направление деятельности парка — организация VR-развлечений для посетителей, проведение игровых сеансов и предоставление доступа к различным виртуальным играм и сценариям.

Для повышения удобства взаимодействия посетителей с игровыми сервисами используются цифровые решения, одним из которых является приложение для сенсорного киоска. Оно позволяет пользователям ознакомиться с доступными играми, выбрать подходящий вариант и получить необходимую информацию без обращения к персоналу.

Организационная структура парка виртуальной реальности включает следующие основные элементы:

Администрация парка виртуальной реальности:

− осуществляет общее руководство деятельностью развлекательного центра;

− координирует работу сотрудников и игровых зон;

− отвечает за развитие инфраструктуры парка и внедрение новых технологий.

Игровые зоны виртуальной реальности:

− обеспечивают проведение игровых сеансов для посетителей;

− включают оборудование виртуальной реальности и игровые станции;

− предоставляют доступ к различным игровым сценариям и режимам.

Персонал парка:

− помогает посетителям подготовиться к игровому сеансу;

− консультирует по выбору VR-игр и игровых режимов;

− следит за исправностью оборудования и безопасностью пользователей.

Посетители парка:

− используют оборудование виртуальной реальности для участия в игровых сеансах;

− выбирают игры и режимы в зависимости от своих предпочтений;

− взаимодействуют с персоналом и цифровыми сервисами парка.

Сенсорный киоск парка виртуальной реальности:

− предоставляет пользователям удобный интерфейс для просмотра каталога VR-игр;

− позволяет получать информацию о доступных играх и их особенностях;

− упрощает процесс выбора развлечений и снижает нагрузку на персонал.

2.2 Разработка дополнительного функционала

В пользовательском интерфейсе приложения была реализована функция изменения языка отображаемого текста. Данная функция позволяет пользователю самостоятельно выбрать язык интерфейса в зависимости от его предпочтений, что особенно важно для посетителей парка виртуальной реальности WARPOINT Veer Mall с разной языковой подготовкой. Переключение языка обеспечивает изменение всех текстовых элементов приложения, включая названия кнопок, элементы поиска, описания VR-игр, жанры и вспомогательные надписи, без необходимости перезапуска приложения.

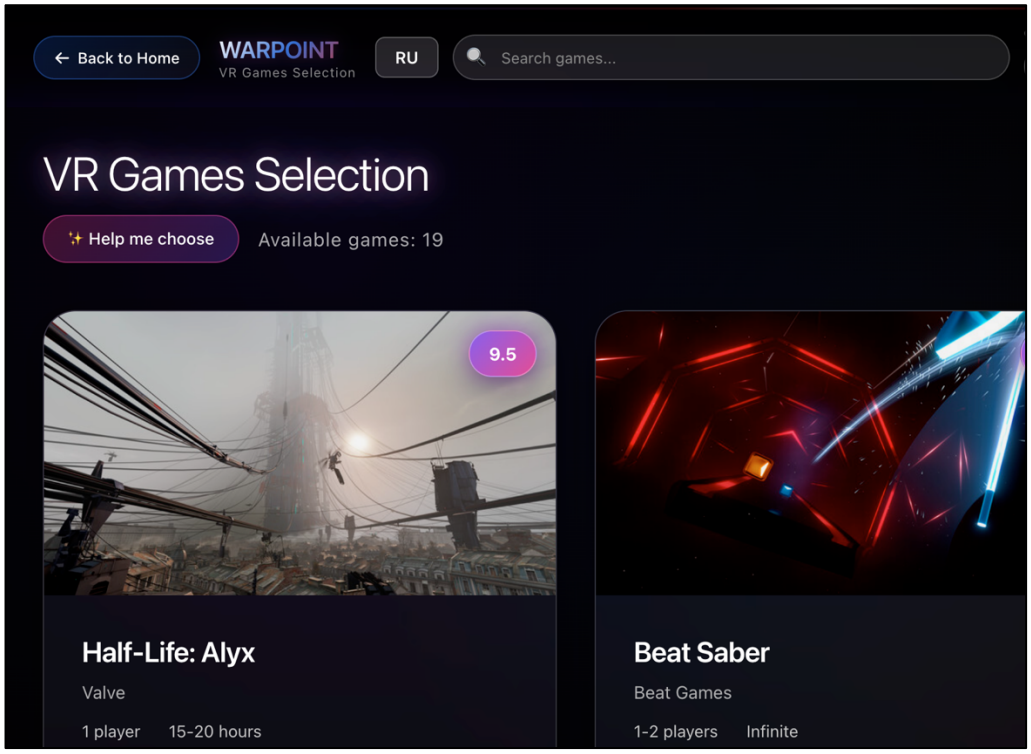

Рисунок 2.1 – Страница веб-приложения с измененным языком

Интерфейс изменения языка, представленный на изображении, реализован в виде интерактивной кнопки переключения языка, расположенной в верхней части экрана приложения. Кнопка содержит обозначения доступных языков, например «RU» и «EN», и позволяет пользователю в любой момент изменить язык отображения интерфейса. Нажатие на соответствующее обозначение инициирует процесс смены текущего языка приложения.

При выборе нового языка система обновляет состояние языкового параметра и сохраняет его в локальном хранилище, что обеспечивает сохранение выбранного языка при последующих запусках приложения. После этого все текстовые элементы интерфейса автоматически перерисовываются с использованием данных из словаря переводов. Кнопки навигации, поля поиска, заголовки, описания VR-игр, жанровые теги и информационные сообщения отображаются на выбранном языке без задержек и визуальных искажений.

Особое внимание уделено корректности и полноте перевода. Для каждого текстового элемента используется уникальный ключ, по которому подбирается соответствующее значение из словаря. В случае отсутствия перевода применяется значение по умолчанию, что предотвращает появление пустых или некорректных надписей в интерфейсе.

Функция изменения языка реализована как отдельный логический модуль и интегрирована в общую архитектуру приложения. Она не влияет на работу других функций и может быть расширена в дальнейшем за счёт добавления новых языков без существенной переработки интерфейса. Таким образом, реализованная функция обеспечивает удобство использования приложения, повышает его доступность для иностранных посетителей и улучшает общее пользовательское восприятие интерфейса.

2.3 Разработка функции быстрого выбора VR-игры в веб-приложении

В пользовательском интерфейсе приложения была реализована функция быстрого выбора VR-игры. Данная функция предназначена для упрощения и ускорения процесса подбора подходящей игры для посетителей парка виртуальной реальности WARPOINT Veer Mall. Она позволяет пользователю без длительного просмотра каталога получить рекомендации на основе своих предпочтений, что особенно удобно для новых посетителей или групп, которые ещё не определились с выбором.

Интерфейс быстрого выбора игры реализован в виде отдельной кнопки «Быстрый подбор», расположенной в основном экране приложения. При нажатии на данную кнопку открывается отдельное модальное окно с мини-интерактивом, представляющим собой пошаговый опрос пользователя. Опрос состоит из нескольких последовательных этапов, на каждом из которых пользователю предлагается выбрать один из вариантов ответа.

В ходе взаимодействия пользователь последовательно указывает количество игроков, предпочитаемый жанр, уровень сложности игры, определяемый на основе рейтинга, а также желаемую длительность сеанса. Переход между этапами осуществляется с помощью кнопок управления, таких как «Далее» или «Найти», что делает процесс выбора понятным и логичным. Все выбранные параметры сохраняются во внутреннем состоянии модуля до завершения опроса.

После прохождения всех этапов система автоматически выполняет анализ введённых данных и запускает алгоритм подбора игр. Доступные VR-игры

фильтруются по заданным параметрам, включая количество игроков, жанр, сложность, продолжительность и текущую доступность в парке виртуальной реальности WARPOINT Veer Mall. Полученные результаты сортируются по рейтингу, после чего пользователю отображается ограниченный список наиболее подходящих вариантов.

Результаты подбора выводятся в виде карточек игр с краткой информацией. При нажатии на карточку выбранной игры пользователь сразу переходит к экрану с подробным описанием, а окно быстрого подбора автоматически закрывается. В случае отсутствия подходящих игр пользователю выводится соответствующее сообщение с возможностью повторить подбор, изменив параметры.

Функция быстрого выбора реализована как независимый модуль и интегрирована в общую архитектуру приложения. Она не влияет на работу других компонентов системы и может быть расширена в дальнейшем, например за счёт добавления новых параметров подбора или изменения логики рекомендаций. Реализация данного функционала повышает удобство использования приложения, сокращает время выбора VR-игры и улучшает пользовательский опыт посетителей парка виртуальной реальности WARPOINT Veer Mall.

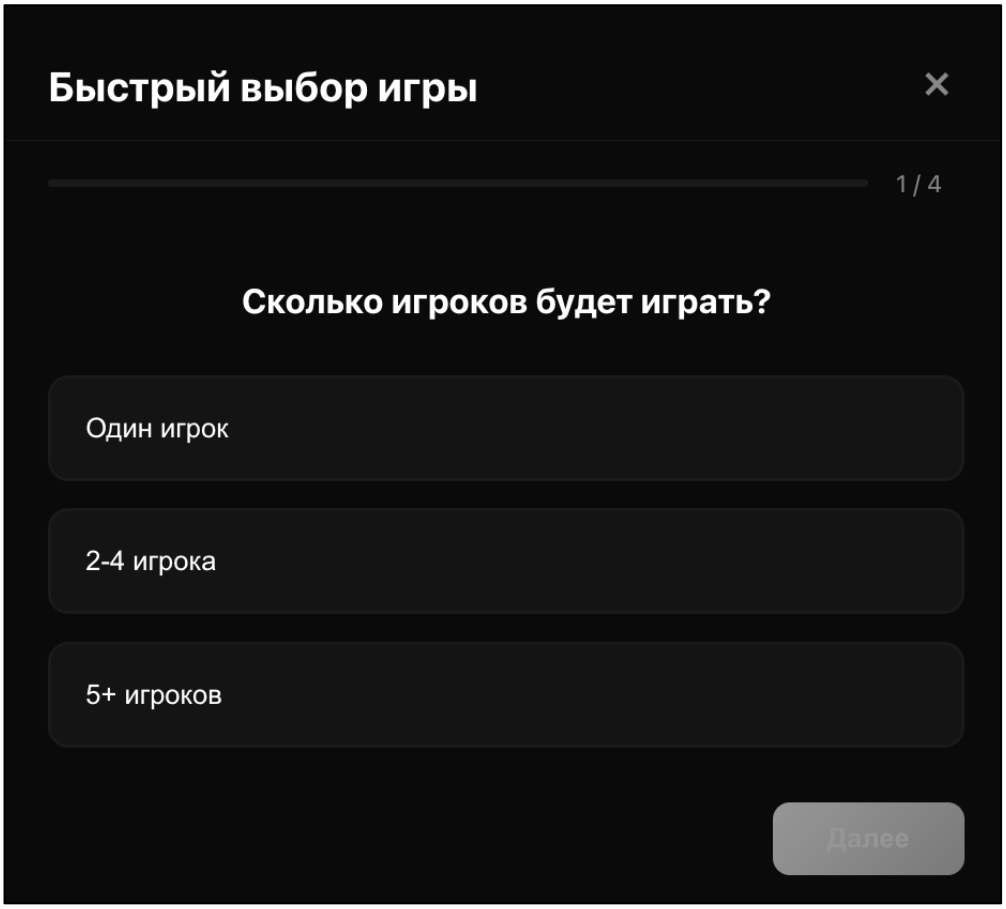

Рисунок 2.2 – Быстрый выбор в веб-приложении

2.4 Разработка функции статистики просмотра VR-игр в веб-приложении

В административной части приложения был реализован модуль статистики, предназначенный для анализа активности посетителей и оценки популярности VR-игр в парке виртуальной реальности WARPOINT Veer Mall. Данный модуль предоставляет администратору наглядную информацию о взаимодействии пользователей с контентом и позволяет принимать обоснованные решения по управлению каталогом игр.

Интерфейс статистики реализован в виде отдельного экрана административной панели, доступ к которому имеют только пользователи с соответствующими правами. При открытии экрана система автоматически загружает сохранённые данные о просмотрах VR-игр, накопленные в процессе работы приложения. Все данные обрабатываются в реальном времени, что обеспечивает актуальность отображаемой информации.

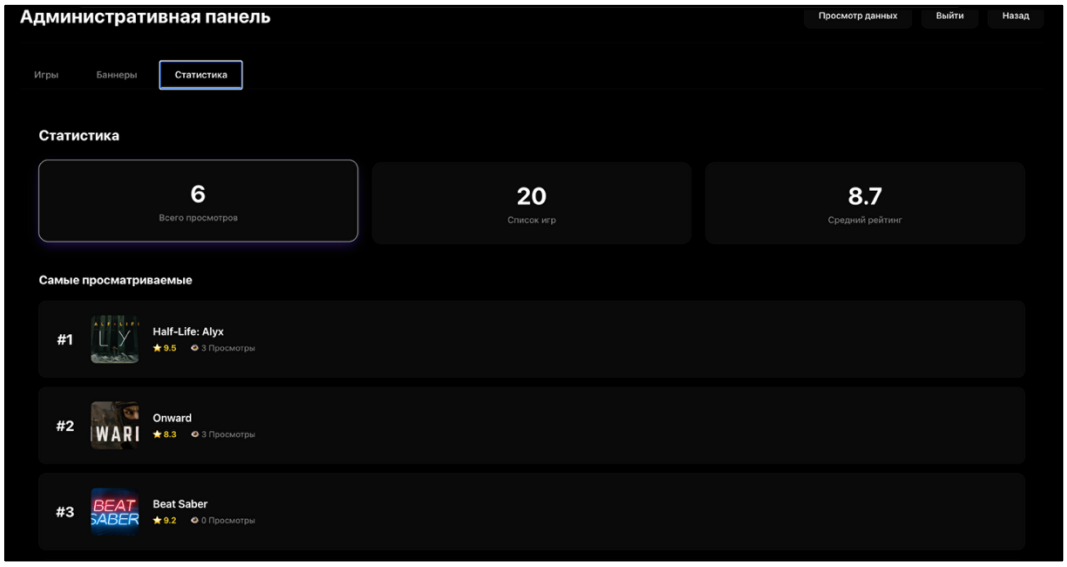

Рисунок 3.3 – Статистика в веб-приложении

В ходе обработки система выполняет подсчёт общего количества просмотров, анализирует частоту просмотров каждой отдельной игры и определяет наиболее популярные позиции. На основе этих данных формируется список VR-игр, которые вызывают наибольший интерес у посетителей. Дополнительно производится расчёт среднего рейтинга, который определяется как отношение суммарного значения рейтингов всех представленных игр к их общему количеству.

Результаты анализа отображаются в административной панели в структурированном и наглядном виде. Администратору предоставляется

информация о количестве просмотров, общем числе доступных VR-игр, среднем рейтинге и списке наиболее популярных игр. Такое представление данных упрощает анализ и позволяет быстро оценить текущую ситуацию.

Модуль статистики интегрирован в общую архитектуру приложения и функционирует независимо от пользовательской части. Его реализация способствует повышению эффективности управления контентом, позволяет своевременно обновлять каталог VR-игр и улучшает качество обслуживания посетителей парка виртуальной реальности WARPOINT Veer Mall.

Заключение

В ходе учебной практики было разработано веб-приложение для автоматизации учета и сопровождения заявок на ремонт климатического оборудования. В процессе выполнения работы была проанализирована предметная область, изучены требования к программному продукту, выбраны подходящие технологии разработки и спроектирована структура базы данных. На основе этого реализовано приложение, обеспечивающее авторизацию пользователей, разграничение прав доступа по ролям, создание, просмотр, редактирование и контроль заявок, а также ведение комментариев, статистики и дополнительных сервисных функций, включая уведомления и QR-код для оценки качества.

Разработанное решение выполнено на Python с использованием Flask, Flask-SQLAlchemy и базы данных SQLite. В результате создана работоспособная информационная система, соответствующая поставленным требованиям. Итогом практики стало закрепление навыков проектирования базы данных, разработки веб-приложений, организации пользовательского интерфейса и документирования программного продукта. Поставленная цель учебной практики была достигнута в полном объеме.

Библиография

1. Соммервилл И. Инженерия программного обеспечения. — М.: Вильямс, 2016. — 704 с.
2. Фаулер М. Рефакторинг. Улучшение существующего кода. — СПб.: Питер, 2019. — 448 с.
3. Нильсен Я. Юзабилити. Практика удобства использования веб-сайтов. — СПб.: Питер, 2017. — 416 с.
4. Pressman R. Software Engineering: A Practitioner’s Approach. — New York: McGraw-Hill, 2014. — 720 с.
5. Sommerville I. Software Engineering. — Boston: Pearson, 2016. — 768 с.
6. Nielsen J. Usability Engineering. — San Diego: Academic Press, 1994. — 362 с.

Приложение А
(справочная)
Листинг программы

Запросы на создание и заполнение базы данных

```
PRAGMA foreign_keys = ON;

CREATE TABLE roles (
  role_id INTEGER NOT NULL,
  role_name VARCHAR(50) NOT NULL,
  PRIMARY KEY (role_id),
  UNIQUE (role_name)
);
CREATE TABLE masters (
  master_id INTEGER NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  specialization VARCHAR(120) NOT NULL,
  phone VARCHAR(30) NOT NULL,
  PRIMARY KEY (master_id)
);
CREATE TABLE clients (
  client_id INTEGER NOT NULL,
  full_name VARCHAR(150) NOT NULL,
  phone VARCHAR(30) NOT NULL,
  email VARCHAR(120),
  address VARCHAR(255),
  PRIMARY KEY (client_id)
);
CREATE TABLE fault_types (
  fault_type_id INTEGER NOT NULL,
  fault_name VARCHAR(120) NOT NULL,
  PRIMARY KEY (fault_type_id),
  UNIQUE (fault_name)
);
CREATE TABLE users (
  user_id INTEGER NOT NULL,
  login VARCHAR(60) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role_id INTEGER NOT NULL,
  master_id INTEGER,
  PRIMARY KEY (user_id),
  UNIQUE (login),
  FOREIGN KEY(role_id) REFERENCES roles (role_id),
  FOREIGN KEY(master_id) REFERENCES masters (master_id)
);
CREATE TABLE equipment (
  equipment_id INTEGER NOT NULL,
  model VARCHAR(120) NOT NULL,
  serial_number VARCHAR(80) NOT NULL,
  client_id INTEGER NOT NULL,
```

```
        equipment_type VARCHAR(80) NOT NULL,
        PRIMARY KEY (equipment_id),
        UNIQUE (serial_number),
        FOREIGN KEY(client_id) REFERENCES clients (client_id)
      );
      CREATE TABLE requests (
        request_id INTEGER NOT NULL,
        create_date DATE NOT NULL,
        description TEXT NOT NULL,
        equipment_id INTEGER NOT NULL,
        master_id INTEGER,
        status VARCHAR(30) NOT NULL,
        completion_date DATE,
        deadline_date DATE,
        customer_approved_extension BOOLEAN NOT NULL,
        PRIMARY KEY (request_id),
        CONSTRAINT  chk_request_status  CHECK  (status  IN  ('open',
'in_progress', 'waiting_parts', 'completed')),
        FOREIGN KEY(equipment_id) REFERENCES equipment (equipment_id),
        FOREIGN KEY(master_id) REFERENCES masters (master_id)
      );
      CREATE TABLE request_details (
        detail_id INTEGER NOT NULL,
        request_id INTEGER NOT NULL,
        fault_type_id INTEGER NOT NULL,
        PRIMARY KEY (detail_id),
        CONSTRAINT     uq_request_fault     UNIQUE     (request_id,
fault_type_id),
        FOREIGN KEY(request_id) REFERENCES requests (request_id),
        FOREIGN     KEY(fault_type_id)     REFERENCES     fault_types
(fault_type_id)
      );
      CREATE TABLE request_comments (
        comment_id INTEGER NOT NULL,
        request_id INTEGER NOT NULL,
        user_id INTEGER NOT NULL,
        comment_text TEXT NOT NULL,
        created_at DATETIME NOT NULL,
        PRIMARY KEY (comment_id),
        FOREIGN KEY(request_id) REFERENCES requests (request_id),
        FOREIGN KEY(user_id) REFERENCES users (user_id)
      );
      CREATE TABLE ordered_parts (
        part_id INTEGER NOT NULL,
        request_id INTEGER NOT NULL,
        part_name VARCHAR(150) NOT NULL,
        quantity INTEGER NOT NULL,
        notes VARCHAR(255),
        ordered_at DATETIME NOT NULL,
        PRIMARY KEY (part_id),
        CONSTRAINT chk_part_quantity CHECK (quantity > 0),
        FOREIGN KEY(request_id) REFERENCES requests (request_id)
      );
```

```
CREATE TABLE status_notifications (
  notification_id INTEGER NOT NULL,
  request_id INTEGER NOT NULL,
  user_id INTEGER NOT NULL,
  old_status VARCHAR(30) NOT NULL,
  new_status VARCHAR(30) NOT NULL,
  message VARCHAR(255) NOT NULL,
  created_at DATETIME NOT NULL,
  PRIMARY KEY (notification_id),
  FOREIGN KEY(request_id) REFERENCES requests (request_id),
  FOREIGN KEY(user_id) REFERENCES users (user_id)
);
CREATE TABLE request_collaborators (
  collaboration_id INTEGER NOT NULL,
  request_id INTEGER NOT NULL,
  master_id INTEGER NOT NULL,
  added_at DATETIME NOT NULL,
  PRIMARY KEY (collaboration_id),
  CONSTRAINT   uq_request_master_collab   UNIQUE   (request_id,
master_id),
  FOREIGN KEY(request_id) REFERENCES requests (request_id),
  FOREIGN KEY(master_id) REFERENCES masters (master_id)
);
CREATE TABLE feedback_reviews (
  review_id INTEGER NOT NULL,
  request_id INTEGER,
  rating INTEGER NOT NULL,
  feedback_text VARCHAR(255),
  created_at DATETIME NOT NULL,
  PRIMARY KEY (review_id),
  CONSTRAINT chk_feedback_rating CHECK (rating >= 1 AND rating
<= 5),
  FOREIGN KEY(request_id) REFERENCES requests (request_id)
);

INSERT INTO roles VALUES(1,'admin');
INSERT INTO roles VALUES(2,'operator');
INSERT INTO roles VALUES(3,'master');
INSERT INTO roles VALUES(4,'manager');
INSERT      INTO      masters      VALUES(1,'Иванов      Сергей
Петрович','Кондиционеры','+7-900-000-11-01');
INSERT      INTO      masters      VALUES(2,'Петрова      Ольга
Викторовна','Вентиляция','+7-900-000-11-02');
INSERT      INTO      masters      VALUES(3,'Сидоров      Алексей
Николаевич','Отопление','+7-900-000-11-03');
INSERT      INTO      masters      VALUES(4,'Артемьев      Денис
Павлович','Электроника','+7-900-000-11-04');
INSERT  INTO  clients  VALUES(1,'Демо  Клиент','+7-900-123-45-
67','demo@example.com','Екатеринбург, ул. Пример, 1');
INSERT INTO clients VALUES(2,'Смирнов Дмитрий Олегович','+7-912-
101-00-01','smirnov@example.com','Екатеринбург, ул. Щорса, 15');
```

```
INSERT INTO clients VALUES(3,'Крылова Алина Сергеевна','+7-912-101-00-02','krylova@example.com','Екатеринбург, ул. Луначарского, 12');
INSERT INTO clients VALUES(4,'ООО АльфаСтрой','+7-912-101-00-03','office@alfastroy.ru','Екатеринбург, ул. Машинная, 38');
INSERT INTO clients VALUES(5,'ПАО СеверТех','+7-912-101-00-04','support@severtech.ru','Екатеринбург, ул. Куйбышева, 44');
INSERT INTO clients VALUES(6,'ИП Лаврентьев','+7-912-101-00-05','lavr@example.com','Екатеринбург, ул. Белинского, 66');
INSERT INTO clients VALUES(7,'Медцентр Вектор','+7-912-101-00-06','service@vectormed.ru','Екатеринбург, ул. Репина, 91');
INSERT INTO clients VALUES(8,'ТЦ Орион','+7-912-101-00-07','tech@orionmall.ru','Екатеринбург, ул. Вайнера, 9');
INSERT INTO fault_types VALUES(1,'Утечка хладагента');
INSERT INTO fault_types VALUES(2,'Неисправность компрессора');
INSERT INTO fault_types VALUES(3,'Сбой электроники');
INSERT INTO fault_types VALUES(4,'Засорение фильтра');
INSERT INTO fault_types VALUES(5,'Шум и вибрация');
INSERT INTO fault_types VALUES(6,'Перегрев платы управления');
INSERT INTO fault_types VALUES(7,'Неисправность датчика температуры');
INSERT INTO users VALUES(1,'admin','scrypt:32768:8:1$WmlOQ9i2a448oCMR$dc483b5cea8a3857dbe20a73d50b1f5c438a410b8abed73e2f39706fd78588906419454f821a270706b8fb0131ea39f36cd035c14a3deddb0ff081a2b27043fc',1,NULL);
INSERT INTO users VALUES(2,'operator','scrypt:32768:8:1$7l9rcAVTlrNOgSPk$17133e38bc20cc89cd7a43348ee7576a04e4822e8d500dd0ed795a7d9e72c3d980e41312eb6278cc03ba74497fb7813429da7f4f79adbb9b68326a3226df2778',2,NULL);
INSERT INTO users VALUES(3,'master','scrypt:32768:8:1$UqAICt4U7VoTHyAv$c611bdd3bf13339796aa9938807f019574ced6abecf29ea14dd94b35683b3ed15ef0375d3f1f9583979983b0e20b0951d21f3b067f55833709d37b077b748837',3,NULL);
INSERT INTO users VALUES(4,'manager','scrypt:32768:8:1$V4CYzMflYGfKIJuk$d28f7e0a32fd18cf56e5021252dd4f50d4430b807e1956d7347aae975a58568b498868ff7af19f55f194683222de506a6421c98e66f36c7c89e974d19a2a6db8',4,1);
INSERT INTO equipment VALUES(1,'Daikin FTXM35','SN-DEMO-001',1,'Кондиционер');
INSERT INTO equipment VALUES(2,'Mitsubishi Electric MSZ-LN35','SN-EKB-1001',2,'Кондиционер');
INSERT INTO equipment VALUES(3,'Systemair SAVE VTR 300','SN-EKB-1002',3,'Вентиляция');
INSERT INTO equipment VALUES(4,'Viessmann Vitocal 100','SN-EKB-1003',4,'Тепловой насос');
INSERT INTO equipment VALUES(5,'Ballu BSVP-18HN1','SN-EKB-1004',5,'Кондиционер');
INSERT INTO equipment VALUES(6,'Komfovent Domekt R 400','SN-EKB-1005',6,'Вентиляция');
INSERT INTO equipment VALUES(7,'LG Deluxe Pro','SN-EKB-1006',7,'Кондиционер');
INSERT INTO equipment VALUES(8,'Baxi Luna 3','SN-EKB-1007',8,'Отопление');
```

```
INSERT INTO equipment VALUES(9,'Toshiba Shorai Edge','SN-EKB-1008',4,'Кондиционер');
INSERT INTO equipment VALUES(10,'Haier Flexis','SN-EKB-1009',3,'Кондиционер');
INSERT INTO requests VALUES(1,'2026-03-05','Периодически отключается при запуске',1,1,'completed','2026-03-05',NULL,0);
INSERT INTO requests VALUES(2,'2026-02-17','Слабое охлаждение в дневное время',2,1,'completed','2026-02-25','2026-02-27',0);
INSERT INTO requests VALUES(3,'2026-02-23','Посторонний шум в приточной установке',3,2,'waiting_parts',NULL,'2026-02-27',1);
INSERT INTO requests VALUES(4,'2026-02-27','Не запускается наружный блок',4,4,'in_progress',NULL,'2026-03-02',0);
INSERT INTO requests VALUES(5,'2026-03-02','Ошибка на дисплее E7',5,1,'open',NULL,'2026-03-07',0);
INSERT INTO requests VALUES(6,'2026-02-19','Вентиляция не выходит на номинальную мощность',6,2,'completed','2026-02-24','2026-02-26',0);
INSERT INTO requests VALUES(7,'2026-02-25','Запах гари при работе',7,4,'waiting_parts',NULL,'2026-02-27',1);
INSERT INTO requests VALUES(8,'2026-02-28','Падение давления в отопительном контуре',8,3,'in_progress',NULL,'2026-03-04',0);
INSERT INTO requests VALUES(9,'2026-02-21','Срабатывает защита компрессора',9,1,'completed','2026-02-25','2026-02-27',0);
INSERT INTO requests VALUES(10,'2026-03-03','Не реагирует на пульт управления',10,4,'open',NULL,'2026-03-09',0);
INSERT INTO request_details VALUES(1,1,1);
INSERT INTO request_details VALUES(2,2,2);
INSERT INTO request_details VALUES(3,2,4);
INSERT INTO request_details VALUES(4,3,3);
INSERT INTO request_details VALUES(5,3,5);
INSERT INTO request_details VALUES(6,4,4);
INSERT INTO request_details VALUES(7,5,5);
INSERT INTO request_details VALUES(8,6,6);
INSERT INTO request_details VALUES(9,6,1);
INSERT INTO request_details VALUES(10,7,7);
INSERT INTO request_details VALUES(11,7,2);
INSERT INTO request_details VALUES(12,8,1);
INSERT INTO request_details VALUES(13,9,2);
INSERT INTO request_details VALUES(14,9,4);
INSERT INTO request_details VALUES(15,10,3);
INSERT INTO request_comments VALUES(1,1,1,'хуйня ебанная','2026-03-05 12:37:53.015611');
INSERT INTO request_comments VALUES(2,1,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.783612');
INSERT INTO request_comments VALUES(3,2,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.785082');
INSERT INTO request_comments VALUES(4,2,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.785084');
```

INSERT INTO request_comments VALUES(5,3,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.786094');
INSERT INTO request_comments VALUES(6,3,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.786097');
INSERT INTO request_comments VALUES(7,4,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.786838');
INSERT INTO request_comments VALUES(8,5,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.787437');
INSERT INTO request_comments VALUES(9,6,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.788100');
INSERT INTO request_comments VALUES(10,6,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.788102');
INSERT INTO request_comments VALUES(11,7,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.788835');
INSERT INTO request_comments VALUES(12,7,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.788837');
INSERT INTO request_comments VALUES(13,8,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.789466');
INSERT INTO request_comments VALUES(14,9,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.790107');
INSERT INTO request_comments VALUES(15,9,4,'Контроль качества выполнен, требуется соблюдение сроков выполнения.','2026-03-05 12:44:54.790109');
INSERT INTO request_comments VALUES(16,10,3,'Проведена диагностика, определены первичные причины неисправности.','2026-03-05 12:44:54.790749');
INSERT INTO request_comments VALUES(17,10,1,'проверьте кондеры','2026-03-05 15:59:20.248202');
INSERT INTO ordered_parts VALUES(1,1,'Кулер',1,'на замену старого','2026-03-05 12:36:19.797331');
INSERT INTO ordered_parts VALUES(2,2,'Датчик температуры',1,'Поставка 2-3 рабочих дня','2026-03-05 12:44:54.793783');
INSERT INTO ordered_parts VALUES(3,3,'Датчик температуры',1,'Поставка 2-3 рабочих дня','2026-03-05 12:44:54.794612');
INSERT INTO ordered_parts VALUES(4,6,'Датчик температуры',1,'Поставка 2-3 рабочих дня','2026-03-05 12:44:54.795250');
INSERT INTO ordered_parts VALUES(5,7,'Датчик температуры',1,'Поставка 2-3 рабочих дня','2026-03-05 12:44:54.795835');

```
INSERT INTO ordered_parts VALUES(6,9,'Датчик температуры',1,'Поставка 2-3 рабочих дня','2026-03-05 12:44:54.796392');
INSERT INTO status_notifications VALUES(1,1,1,'in_progress','completed','Заявка №1: статус изменен с ''В процессе'' на ''Завершена''.','2026-03-05 12:38:13.024327');
INSERT INTO status_notifications VALUES(2,2,4,'open','completed','Заявка №2: статус обновлен до ''completed''.','2026-03-05 12:44:54.798485');
INSERT INTO status_notifications VALUES(3,3,4,'open','waiting_parts','Заявка №3: статус обновлен до ''waiting_parts''.','2026-03-05 12:44:54.799190');
INSERT INTO status_notifications VALUES(4,4,4,'open','in_progress','Заявка №4: статус обновлен до ''in_progress''.','2026-03-05 12:44:54.799727');
INSERT INTO status_notifications VALUES(5,6,4,'open','completed','Заявка №6: статус обновлен до ''completed''.','2026-03-05 12:44:54.800241');
INSERT INTO status_notifications VALUES(6,7,4,'open','waiting_parts','Заявка №7: статус обновлен до ''waiting_parts''.','2026-03-05 12:44:54.800736');
INSERT INTO status_notifications VALUES(7,8,4,'open','in_progress','Заявка №8: статус обновлен до ''in_progress''.','2026-03-05 12:44:54.801220');
INSERT INTO status_notifications VALUES(8,9,4,'open','completed','Заявка №9: статус обновлен до ''completed''.','2026-03-05 12:44:54.801758');
INSERT INTO request_collaborators VALUES(1,1,1,'2026-03-05 12:35:56.272769');
INSERT INTO request_collaborators VALUES(2,3,1,'2026-03-05 12:44:54.777460');
INSERT INTO request_collaborators VALUES(3,4,1,'2026-03-05 12:44:54.778827');
INSERT INTO request_collaborators VALUES(4,7,1,'2026-03-05 12:44:54.779490');
INSERT INTO request_collaborators VALUES(5,8,1,'2026-03-05 12:44:54.779993');
INSERT INTO request_collaborators VALUES(6,2,1,'2026-03-05 16:01:41.630818');
INSERT INTO feedback_reviews VALUES(1,1,5,'Работа выполнена качественно и в согласованный срок.','2026-03-05 12:43:47.713560');
INSERT INTO feedback_reviews VALUES(2,2,4,'Работа выполнена качественно и в согласованный срок.','2026-03-05 12:43:47.713567');
INSERT INTO feedback_reviews VALUES(3,6,5,'Работа выполнена качественно и в согласованный срок.','2026-03-05 12:43:47.713569');
INSERT INTO feedback_reviews VALUES(4,9,4,'Работа выполнена качественно и в согласованный срок.','2026-03-05 12:43:47.713570');
```